

PRD: Prama (MVP v1.0)

I. Technical Stack & Environment

- **Frontend:** React 19, Tailwind CSS, Lucide React (Icons).
 - **Backend:** Node.js, Express.js.
 - **Database:** MongoDB (Atlas).
 - **AI:** Gemini 3.1 Pro (via Google Cloud/Vertex AI).
 - **UI Vibe:** "The Calming Ledger" (Background: #FDFCF0, Text: #34495E, Accents: #8FBC8F).
-

II. Data Schema (The "How" of Storage)

To avoid debugging hours, we use a **nested relational-document structure**.

1. Case Schema

JavaScript

```
{
  caseId: String (Unique),
  clientName: String,
  lawyerId: ObjectId,
  baseLayer: {
    startDate: Date,
    endDate: Date,
    location: String,
    jurisdiction: String
  },
  status: Enum ["Collecting", "Synthesized", "Finalized"],
  createdAt: Date
}
```

2. Session & Deposit Schema

Each Case has multiple Sessions. Each Session has multiple Deposits.

JavaScript

```
{
```

```
sessionId: ObjectId,  
caseId: String,  
timestamp: Date,  
deposits: [  
  {  
    depositId: ObjectId, // Essential for Source Tracing  
    content: String (Raw text or transcript),  
    sensoryTag: Enum ["Visual", "Auditory", "Olfactory", "Somatic"],  
    addedAt: Date  
  }  
]  
}
```

III. Core Modules (The "What")

1. Case Command Center

- **What:** Dashboard displaying all active cases.
- **How:** A `GET /api/cases` call populates a card-based UI.
- **Action:** "New Case" button opens a form to fill the `baseLayer` (The skeleton of the case).

2. Live Session Workspace

- **What:** The active capture interface used by the lawyer during meetings.
- **How:** A real-time text input that pushes to the `deposits` array.
- **Key Feature: Sensory Buttons.** Clicking a tag (e.g., "Smell") prepends the metadata to that specific deposit.

3. The Stitching Engine (AI Integration)

- **What:** A "Generate Timeline" button that triggers Gemini.
- **How:** The backend sends **all** deposits from **all** sessions for a specific `caseId` to Gemini.
- **AI Prompting:** Gemini is instructed to return **only** a JSON array of events sorted by "Event Date" (not "Input Date").

4. Source Trace UI

- **What:** A vertical timeline view of the AI's output.
- **How:** Each item in the timeline stores the `depositId` it came from. Clicking an item highlights the original raw text in the sidebar.

IV. AI Orchestration Logic (The "Brain")

To prevent hallucinations and maintain legal integrity, the Gemini prompt is **strict**:

The System Instruction:

"You are a Legal Chronology Expert. You will receive fragmented memory deposits.
Your task:

1. Extract temporal entities (dates/times).
2. Order events chronologically.
3. **DO NOT** add facts not present in the input.
4. If two deposits contradict, flag the event with `isConflict: true`.
5. Output **strictly** in JSON format: `[{ "date": "", "event": "", "sourceId": "", "isConflict": boolean }]."`

VI. User Flow (Direct Path)

1. **Onboarding:** Lawyer creates Case -> Fills Base Layer (Dates/Names).
2. **Meeting:** Lawyer opens "Session 1" -> Types notes or uses Voice-to-Text -> Tags fragments with sensory icons.
3. **Interval:** Days pass. Lawyer opens "Session 2" -> Adds more fragments as the victim remembers.
4. **Synthesis:** Lawyer clicks "Generate Narrative" -> AI stitches Session 1 & 2 -> UI displays a chronological "Statement of Facts."
5. **Validation:** Lawyer clicks an event -> UI shows exactly which note from which session that event came from.
6. **Export:** Lawyer downloads the "Affidavit Draft" as a PDF.